1. Aim

Introduction to Truffle, Establish local blockchain using Truffle and deployment of Smart Contract.

2. Theory

2.1 Blockchain and Transparent Donation Tracking

A blockchain is a distributed, append-only ledger in which transactions are grouped into blocks that are cryptographically linked to the previous block. Once a block is accepted, its records cannot be silently altered. In a charity setting this means every campaign, donation and withdrawal is stored as a time-stamped transaction that anyone with access to the network can inspect. Donors do not have to rely on a central organisation's records alone, which improves transparency and accountability.

2.2 Truffle and Ganache

Truffle is a development framework for Ethereum that provides compilation, migration (deployment) and testing of smart contracts. Ganache, part of the same tool suite, is a personal Ethereum blockchain that runs on the developer's computer. It creates pre-funded test accounts (100 test ETH each by default), mines blocks instantly (automining), and shows blocks, transactions, contracts and logs in a graphical interface. Because the Ether is test Ether with no real value, contracts can be deployed and tested freely and without cost. In this experiment Ganache provided the local blockchain; Remix IDE, rather than a Truffle project, was used to compile and deploy the contract.

2.3 MetaMask

MetaMask is a browser-based wallet. It stores account keys, lets the user add custom networks (for example a local RPC endpoint), displays balances and asks the user to confirm or reject each transaction before it is signed and broadcast. Accounts from Ganache can be imported into MetaMask using their private keys, which is acceptable only for development accounts holding test Ether.

2.4 Remix IDE, Solidity, Compilation and Deployment

Solidity is the main high-level language for Ethereum smart contracts. Remix IDE is a browser-based environment for writing, compiling, deploying and testing Solidity code. Compilation converts the source into EVM bytecode and an ABI (the description of the contract's functions). Deployment sends a contract-creation transaction to the network; once it is mined the contract receives its own address. Remix can talk to a node through an external HTTP provider, which allows it to deploy to a Ganache blockchain.

2.5 Smart Contracts, Transactions, Gas, Wei and Ether

A smart contract is code stored on the blockchain that executes exactly as written when it is called by a transaction. Functions that change state (such as creating a campaign or donating) need a transaction and are mined into a block; read-only *view* functions are called without a transaction and cost no gas. **Gas** is the unit that measures computational work, and the transaction fee equals gas used multiplied by gas price. Ether amounts are handled in **wei**, the smallest unit: 1 ETH = 10^{18} wei. Solidity therefore works with integers such as 1000000000000000000 instead of decimals.

2.6 The Charity Contract

The contract **CharityDonation** stores an `owner`, a running `totalDonations`, a `campaignCount` and a mapping of campaigns. Each campaign (a `struct`) holds a name, a payable beneficiary address, a target amount, the amount raised and an `active` status flag. The functions used in this experiment are:

- `createCampaign(string name, address beneficiary, uint256 target)` - registers a new campaign.
- `getCampaign(uint256 id)` - returns the campaign's name, beneficiary, target, amount raised and active status.
- `donate(uint256 id)` - payable function that records an Ether donation to the given campaign.
- `getDonation(uint256 id, address donor)` - returns the amount a given address has donated to a campaign.
- `withdrawFunds(...)` - releases collected funds to the beneficiary under conditions defined in the contract (see Step 12).

2.7 Traceability Through Transactions

Every state-changing call is a transaction with a unique hash, a sender address, a destination contract address, gas used and an Ether value. The donor's address and amount are also stored in the contract and can be read back with `getDonation`. Together, the transaction list and the contract's stored values provide a verifiable trail of campaign-related activity.

3. Software and Tools Required

Tool	Purpose in this experiment
Ganache (v2.7.1 as shown in the screenshots)	Local Ethereum blockchain with funded test accounts and a transaction viewer.
MetaMask	Wallet used to add the local network and import a Ganache account.
Remix IDE	Editing, compiling, deploying and calling the smart contract.
Solidity (pragma ^0.8.20)	Language in which CharityDonation.sol is written.
Web browser	Runs the MetaMask extension and the browser-based Remix IDE.
Windows computer	Host system on which Ganache was run.

4. Implementation Steps

The steps below follow the workflow shown in the supplied screenshots. Figure numbers refer to Section 5.

Reference values used in this experiment

Item	Value
Contract / source file	CharityDonation / CharityDonation.sol
Campaign name	Education Fund

Item	Value
Target	1000000000000000000 wei = 1 ETH (shown in Figures 16 and 19)
Example conversion	0.1 ETH = 100000000000000000 wei (illustrative; see note below)
Ganache RPC endpoint	http://127.0.0.1:7545 initially (Figures 3, 4); http://127.0.0.1:8545 later (Figures 5, 22)
Network ID shown in Ganache	5777
Gas price shown in Ganache	20000000000 wei (20 gwei)

Wei and Ether. 1 ETH = 1000000000000000000 wei (10^{18}), so the campaign target of 1 ETH is entered as 1000000000000000000. A donation of 0.1 ETH equals 100000000000000000 wei (10^{17}). **Note:** 0.1 ETH is given only as an example of the conversion. The screenshots show 1 ETH transaction values in Ganache (Figure 22) and a total of 2 ETH recorded for the donor (Figures 18 and 19); no 0.1 ETH donation appears in the evidence.

Step 1: Launch Ganache and start the local blockchain.

Open Ganache on the Windows computer. On the start screen choose **New Workspace (Ethereum)** and enter the workspace name **CharityDonationSystem**. No Truffle project is linked (Figures 1-2).

Step 2: Check the RPC endpoint, network ID, accounts and test Ether balances.

In the **Accounts** tab note the RPC server, network ID (**5777**), gas price, gas limit and mining status (automining). Each listed account holds 100 ETH of test Ether (Figure 3). Ganache's RPC server appears as 127.0.0.1:7545 in this first screenshot and as 127.0.0.1:8545 in the later screenshot (Figure 22).

Step 3: Configure the local Ganache network in MetaMask.

In MetaMask open the network settings and add a custom RPC URL pointing to the local Ganache node (Figure 4). The network is then listed as **Localhost 8545** with currency symbol ETH (Figure 5). Import a Ganache account into MetaMask: copy its private key from Ganache's account information dialog (Figure 6), choose **Add wallet - Import an account** (Figure 7), paste the key (Figure 8) and confirm that **Imported Account 1** appears (Figure 9). MetaMask then shows the account on Localhost 8545 with 100 ETH (Figure 10).

Step 4: Open Remix IDE and connect it to the Ganache network.

In Remix open **Deploy & Run Transactions** and set the environment to **Custom - External Http Provider**. The account shown (0xa8c...388e4) is the same account that was imported in MetaMask and it holds 100 ETH (Figure 11).

Step 5: Write or open CharityDonation.sol.

Create or open the file `CharityDonation.sol` in the Remix file explorer. The contract uses the licence identifier MIT and `pragma solidity ^0.8.20` (Figure 12).

Step 6: Compile the Solidity smart contract.

Use the Solidity compiler tab (or Ctrl+S) to compile the file. Remix shows a green **Compiled** indicator in the editor toolbar when compilation succeeds (Figure 12).

Step 7: Deploy the contract and confirm the transaction.

Click **Deploy** in Deploy & Run Transactions. The contract appears under **Deployed Contracts** (Figure 13) and Ganache lists a **Contract Creation** transaction with the new contract address (Figure 14). The screenshots do not include a MetaMask confirmation pop-up.

Step 8: Create the Education Fund campaign.

Expand `createCampaign` and enter the campaign name **Education Fund**, a beneficiary address (the Ganache account at index 1, 0x1fD35a...fBd3) and the target in wei (`1000000000000000000` for 1 ETH), then press **Transact** (Figure 15). Success is verified in Step 9.

Step 9: Read campaign information using getCampaign.

Enter campaign ID `1` in `getCampaign` and press **Call**. It returns the name, beneficiary, target, amount raised and active status (Figure 16).

Step 10: Make a test donation using donate and verify with getDonation.

Enter campaign ID `1` in `donate`, set the Ether amount in Remix's Value field (for example 0.1 ETH = 100000000000000000 wei), and press **Transact** (Figure 17). Then call `getDonation(1, donor address)` (Figures 17-18) and re-read the campaign with `getCampaign` (Figure 19). The public variables `totalDonations` and `campaignCount` can also be read (Figures 20-21).

Step 11: Inspect contract-related transactions in Ganache.

Open the **Transactions** tab in Ganache to see each contract call with its transaction hash, sender, contract address, gas used and value (Figure 22). In MetaMask the account balance reflects the Ether spent (Figure 23).

Step 12: Explain the withdrawal function and its conditions.

The contract's fund-release function (named `withdrawFunds` in the experiment brief) transfers collected Ether to the campaign's beneficiary. Contracts of this type normally restrict withdrawal, for example to an authorised caller (owner or beneficiary), to existing campaigns that hold funds, and sometimes to campaigns whose target has been reached or that are still active. **The withdrawal function was not executed in the supplied screenshots, and the contract source visible in Figure 12 ends at line 18, so the exact conditions must be confirmed from the full source code.** This report therefore makes no claim that a withdrawal took place.

5. Implementation Screenshots and Observations

Screenshots were extracted from the supplied document at their original resolution and are shown unaltered, in the order of the implementation steps. Thin grey borders were added only to separate them from the page.

Step 1: Launch Ganache and start the local blockchain

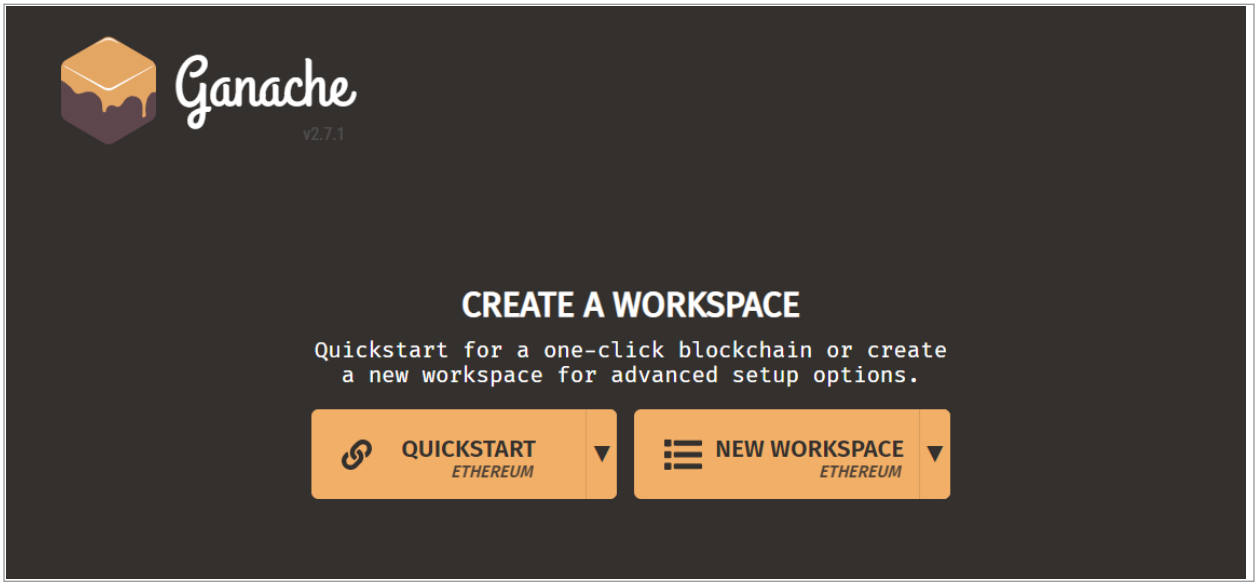


Figure 1: Ganache start screen (Create a Workspace).

Observation: Ganache v2.7.1 is open and offers *Quickstart* and *New Workspace* for Ethereum. This confirms Ganache was launched on the local machine.



Figure 2: Ganache workspace configuration.

Observation: The workspace is named **CharityDonationSystem**. The Truffle Projects list is empty, so no Truffle project was linked; the contract was handled through Remix IDE.

Step 2: Check the RPC endpoint, network ID, accounts and balances

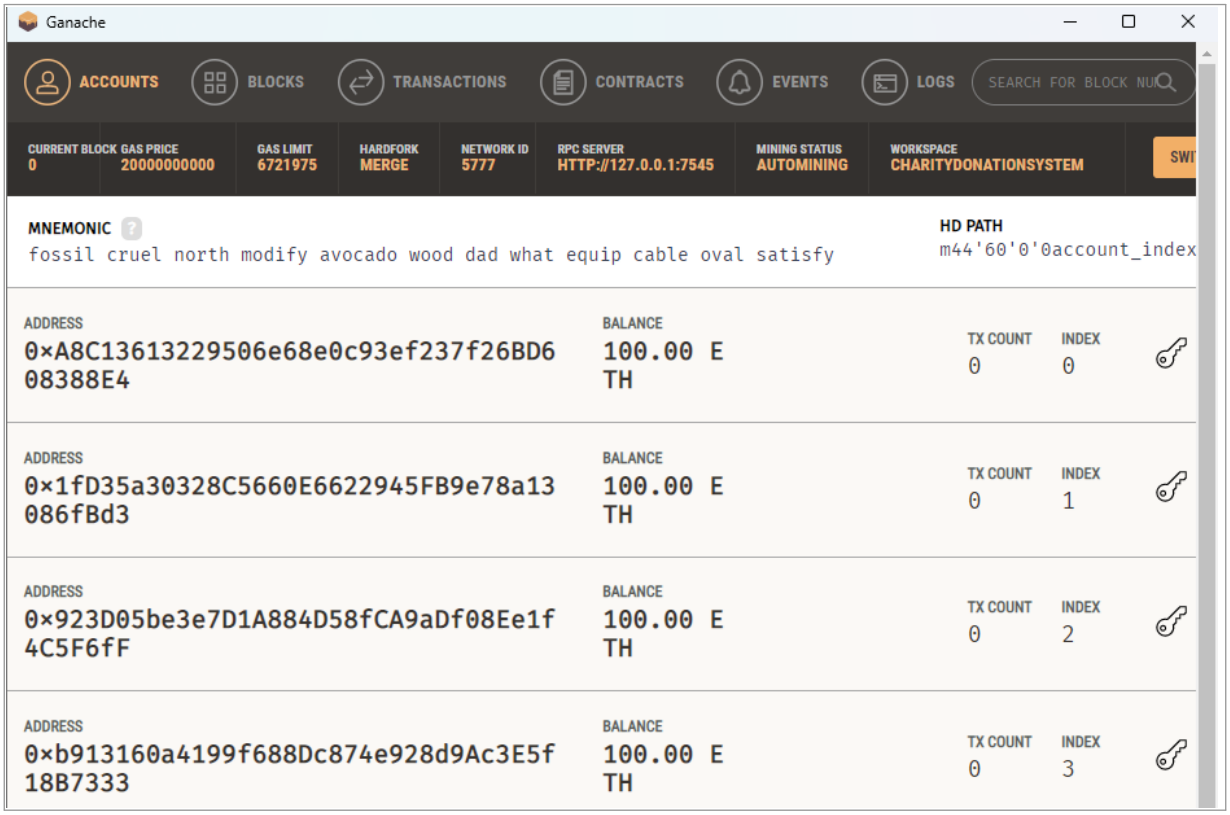


Figure 3: Ganache local blockchain and account balances.

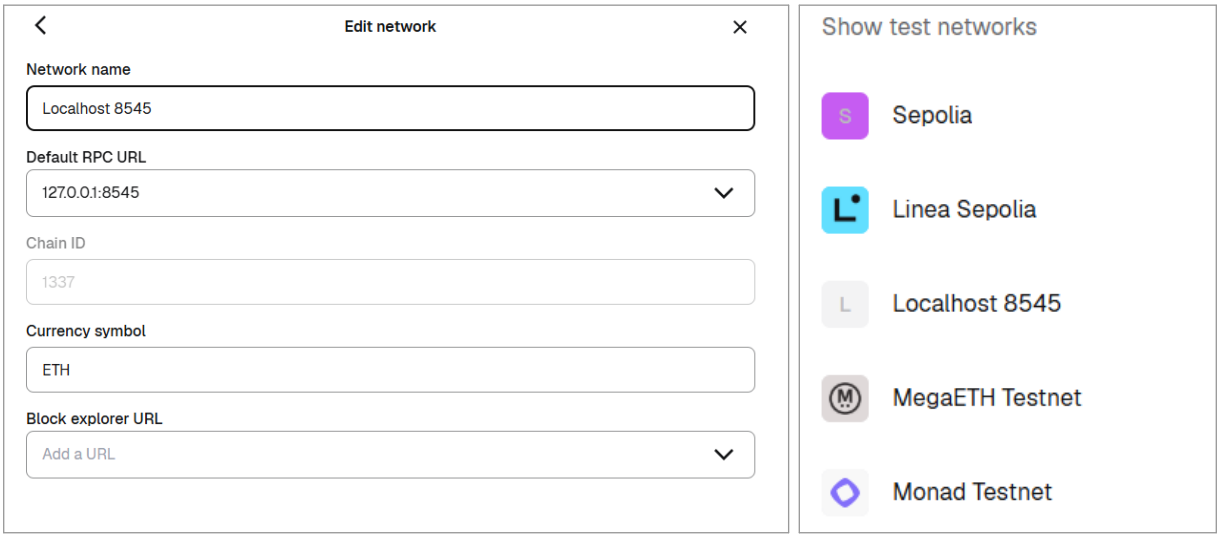
Observation: The status bar shows current block 0, gas price 20000000000, gas limit 6721975, hardfork Merge, **network ID 5777**, RPC server **HTTP://127.0.0.1:7545**, automining, and workspace CHARITYDONATIONSYSTEM. The visible accounts each hold 100.00 ETH with 0 transactions. Index 0 is 0xA8C1...88E4 and index 1 is 0x1fD35a...fBd3 (later used as the campaign beneficiary).

Step 3: Configure the local Ganache network in MetaMask and import an account



Figure 4: MetaMask Add RPC URL form.

Observation: The RPC URL **http://127.0.0.1:7545** is entered, matching Ganache's RPC server in Figure 3; the optional name is blank.



(a) Edit network *(b) Network list*

Figure 5: MetaMask network configuration for the local Ganache node.

Observation: (a) The network is named **Localhost 8545** with default RPC URL 127.0.0.1:8545 and currency symbol ETH; the greyed Chain ID field shows 1337. (b) The network list includes Localhost 8545. Note that Ganache's *network ID* (5777) and the *chain ID* used by wallets (1337) are different identifiers. The port 8545 here matches the later Ganache screenshot (Figure 22) rather than the 7545 in Figure 3.

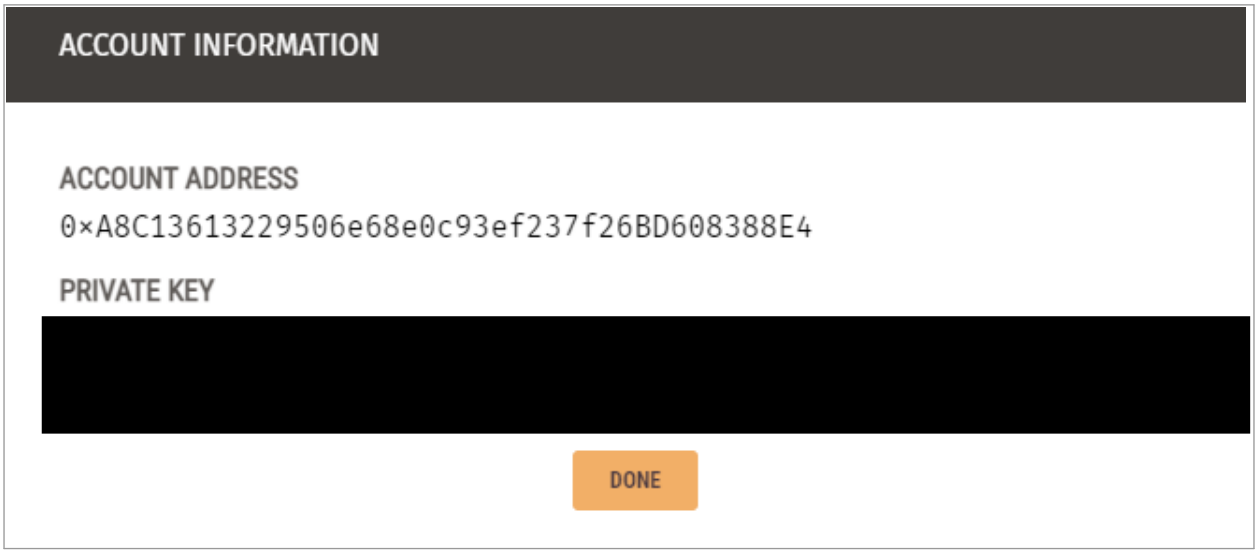
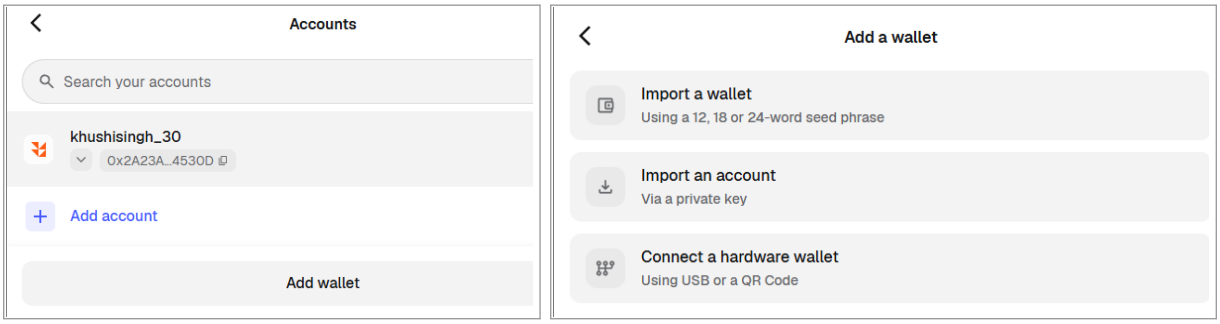


Figure 6: Ganache account information (address and private key).

Observation: Account 0xA8C13613229506e68e0c93ef237f26BD608388E4 (index 0) with its private key, used to import the account into MetaMask. Ganache's own warning applies: this is a development key and must never be used on a public blockchain.



(a) Accounts list *(b) Add a wallet menu*

Figure 7: MetaMask accounts and the Add a wallet options.

Observation: (a) Before import, MetaMask lists only the account khushisingh_30. (b) The Add a wallet menu offers seed phrase import, private-key import and hardware wallet; *Import an account via a private key* is the option for a Ganache account.

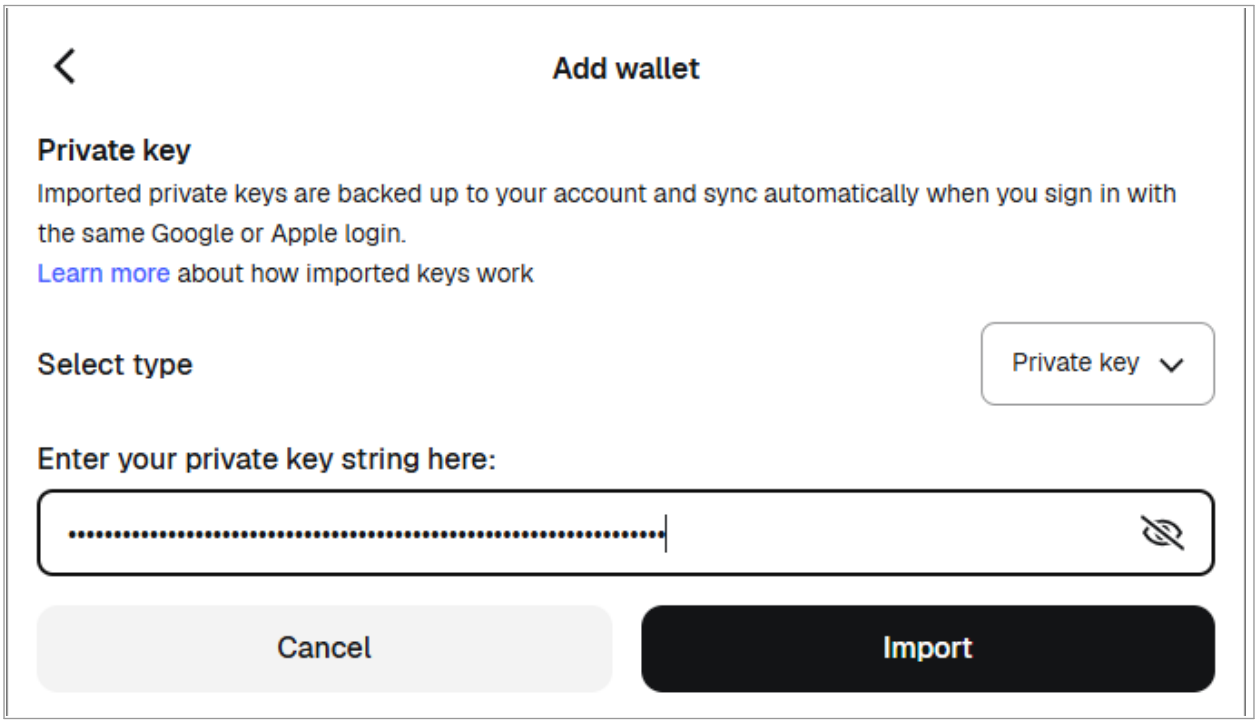


Figure 8: Importing the Ganache account by private key.

Observation: The private-key field is masked and the **Import** button is ready to be pressed, showing the import procedure.

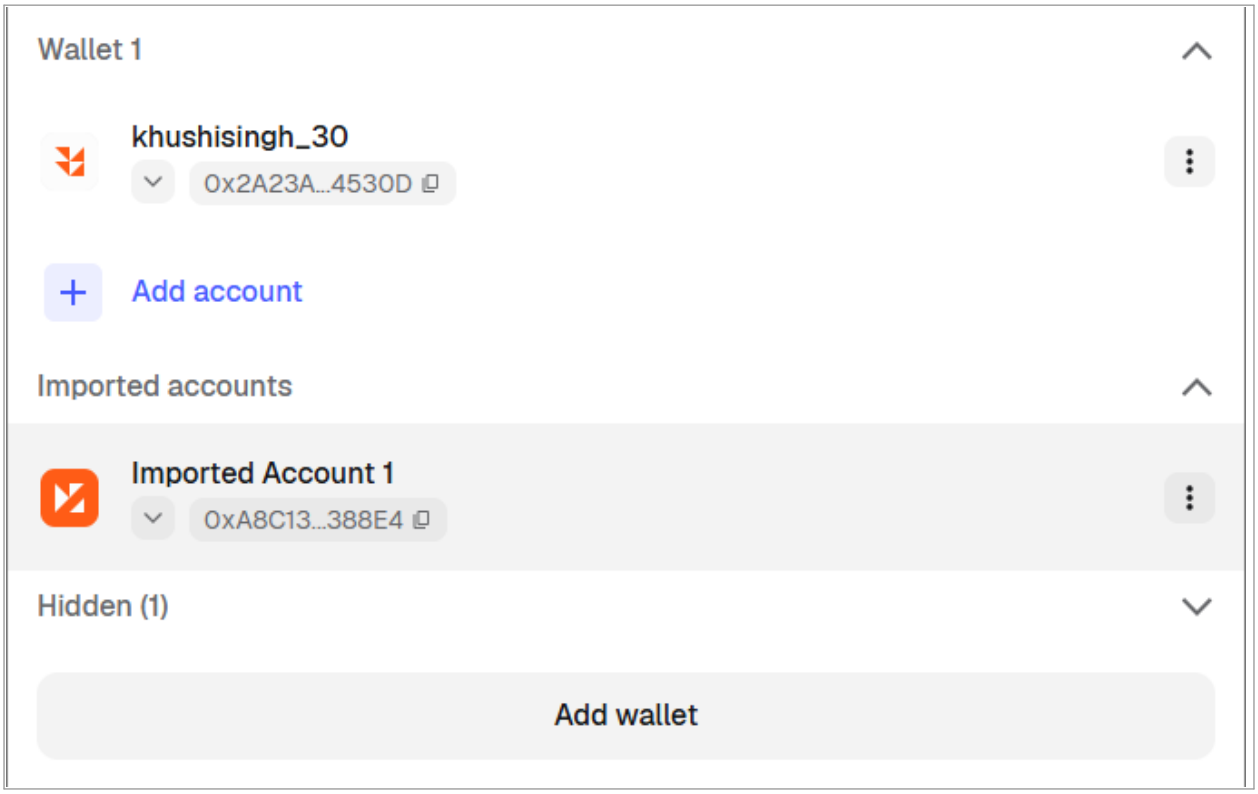


Figure 9: Imported account listed in MetaMask.

Observation: Imported Account 1 (0xA8C13...388E4) now appears under Imported accounts. The address matches Ganache account index 0.

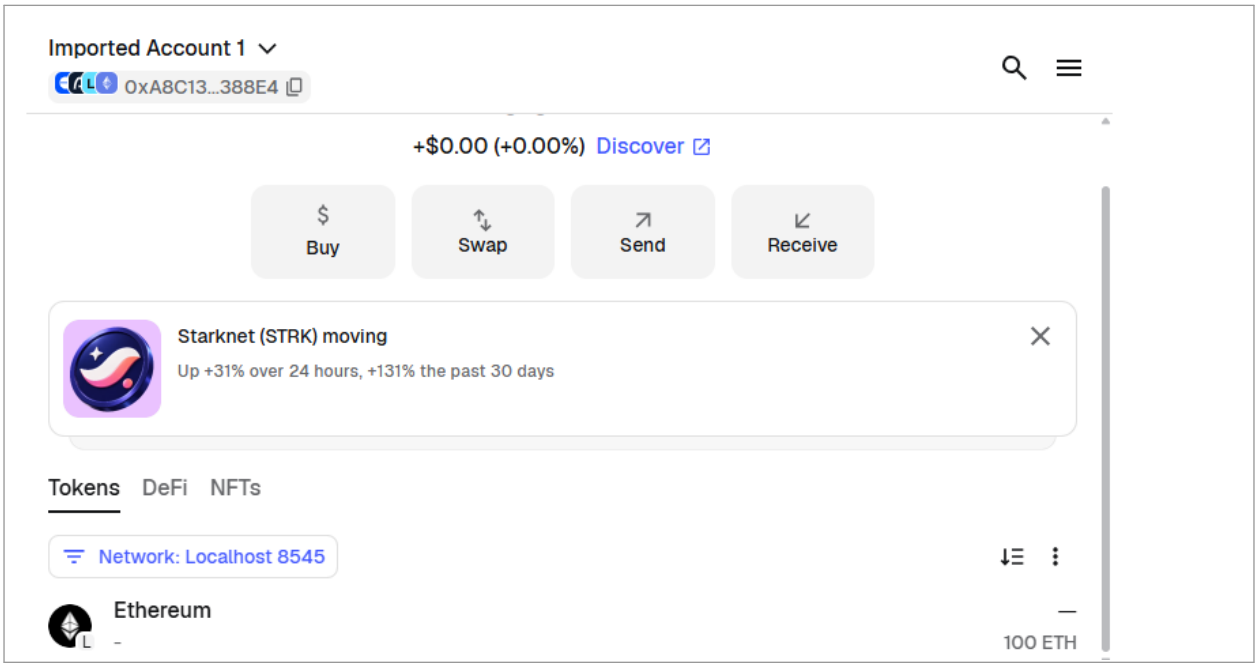


Figure 10: MetaMask connected to the local Ganache network.

Observation: Imported Account 1 is shown with the network filter **Localhost 8545** and a balance of **100 ETH**, matching the balance of the same account in Ganache.

Step 4: Open Remix IDE and connect it to the Ganache network

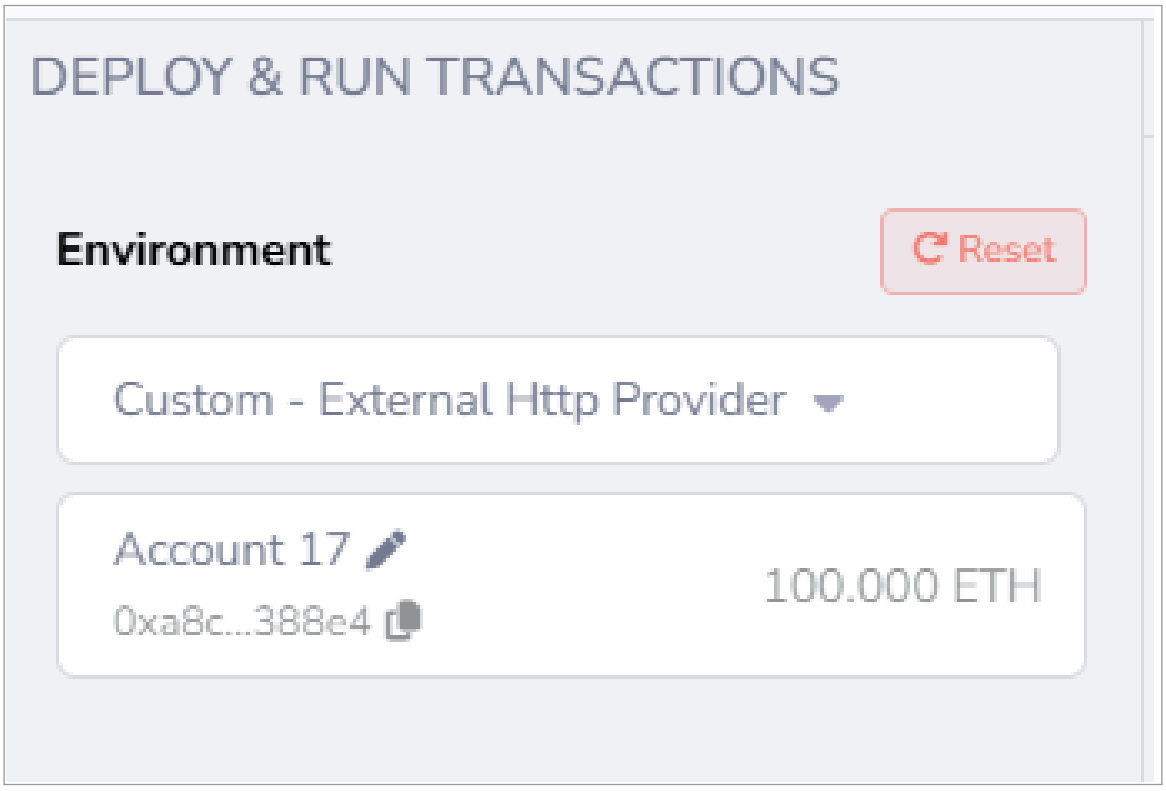


Figure 11: Remix Deploy & Run Transactions connected to the local node.

Observation: The environment is **Custom - External Http Provider** and the selected account (0xa8c...388e4) shows 100.000 ETH, the same account as in Ganache and MetaMask. This indicates Remix was connected to the local blockchain.

Step 5-6: Write or open CharityDonation.sol and compile it



Figure 12: Solidity smart contract in Remix IDE with compilation status.

Observation: The editor tab *CharityDonation.sol* is open and the green **Compiled** indicator is visible. Visible code (lines 1-18): MIT licence, `pragma solidity ^0.8.20`, `contract CharityDonation` with state variables `owner`, `totalDonations`, `campaignCount`, the `Campaign` struct (name, payable beneficiary, target, raised, active) and the `campaigns` mapping. The remaining functions are not visible in this screenshot.

Step 7: Deploy the contract and confirm the transaction

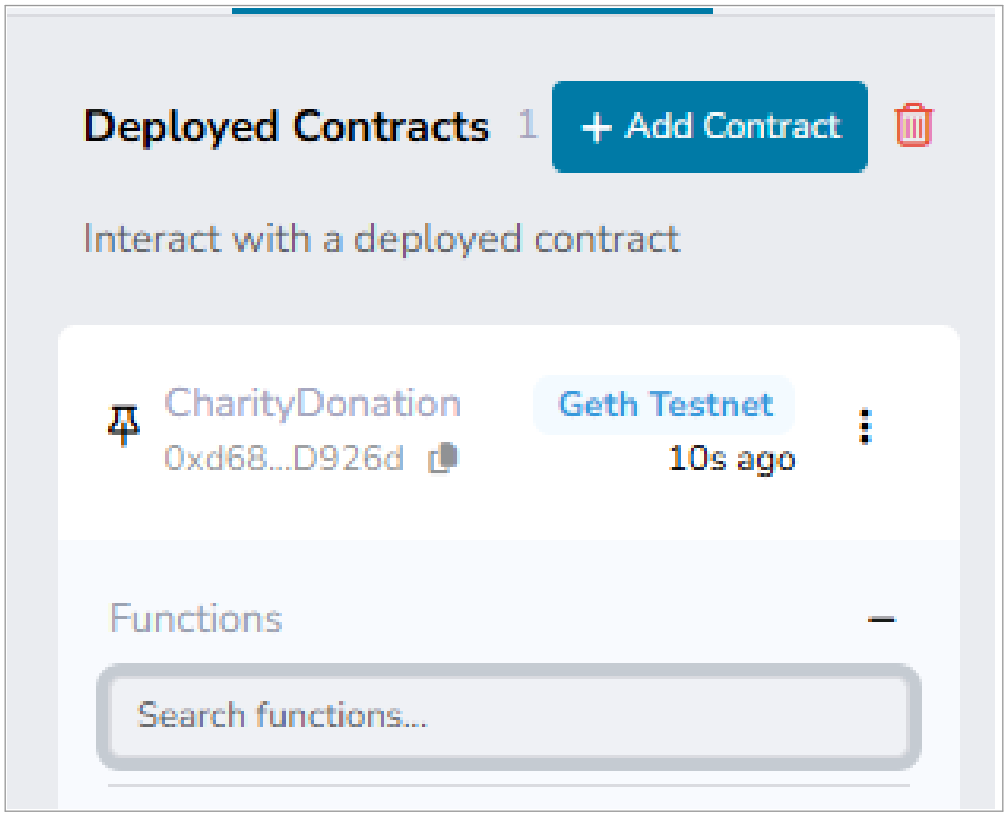


Figure 13: Deployed contract listed in Remix.

Observation: Under Deployed Contracts, **CharityDonation** at 0xd68...D926d appears (deployed 10 s earlier, labelled Geth Testnet by Remix) with its function list available for interaction.



Figure 14: Contract creation transaction in Ganache.

Observation: Ganache shows a **CONTRACT CREATION** transaction (hash beginning 0x3011feca...) from 0xA8C13613...88E4, creating contract 0xd68cE8eD0CD35FDF477403A1a02a5363C6dD926d, gas used 1451523, value 0. This confirms that the deployment was recorded on the local blockchain.

Step 8: Create the Education Fund campaign

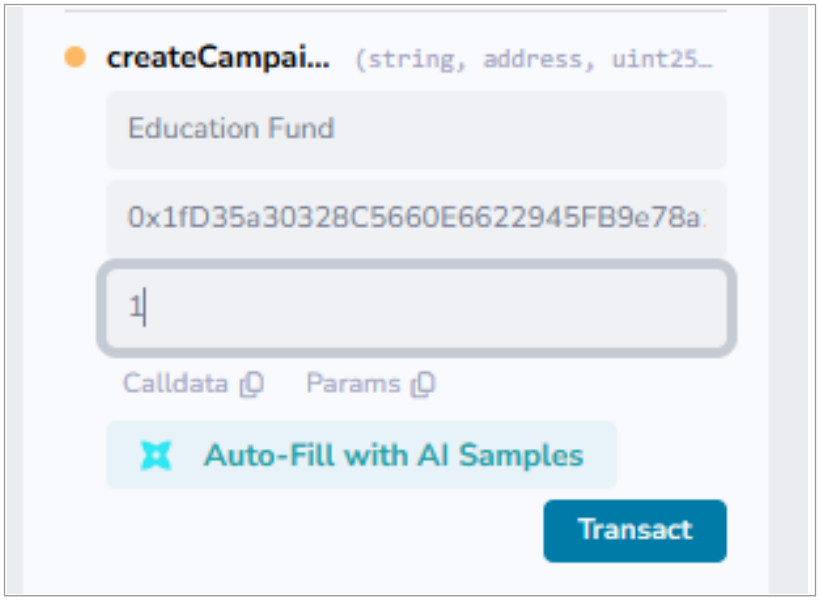


Figure 15: Creation of the Education Fund campaign (createCampaign inputs).

Observation: The function `createCampaign(string, address, uint256)` is filled with the name **Education Fund**, beneficiary address 0x1fD35a30328C5660E6622945FB9e78a... and the target field being typed (it shows 1 with the cursor). The screenshot captures the inputs before pressing Transact; successful creation is evidenced by Figures 16 and 19 and the transaction list in Figure 22.

Step 9: Read campaign information using getCampaign

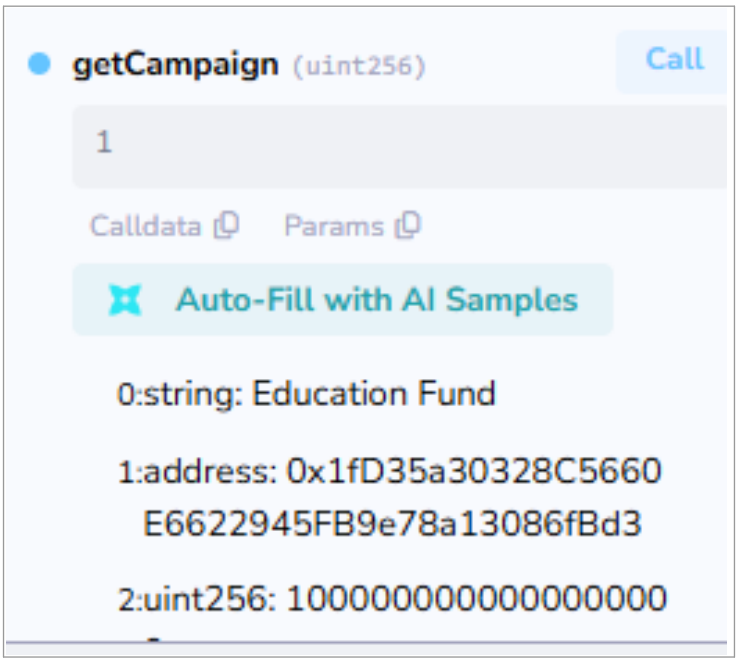
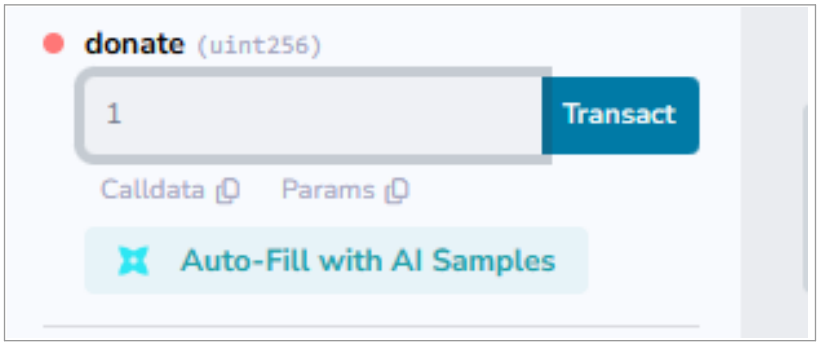


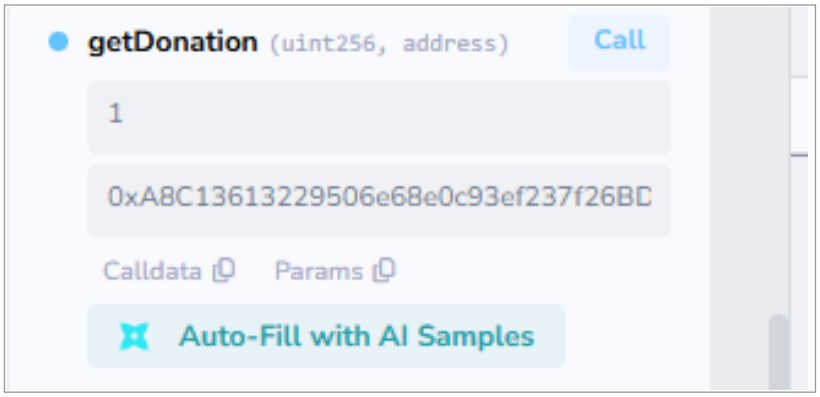
Figure 16: Campaign details returned by getCampaign (partial view).

Observation: Calling `getCampaign(1)` returns the string **Education Fund**, the beneficiary address 0x1fD35a30328C5660E6622945FB9e78a13086fBd3 and a target value beginning 1000... (the rest of the output is cut off in this crop). The full output appears in Figure 19.

Step 10: Make a test donation using donate and verify using getDonation



(a) donate



(b) getDonation inputs

Figure 17: Donation function and getDonation inputs.

Observation: (a) `donate(uint256)` with campaign ID 1 and the Transact button; it is shown in red, which Remix uses for payable functions. The Ether amount set in the Value field is not visible in this screenshot. (b) `getDonation(uint256, address)` with campaign ID 1 and donor address 0xA8C13613229506e68e0c93ef237f26BD...

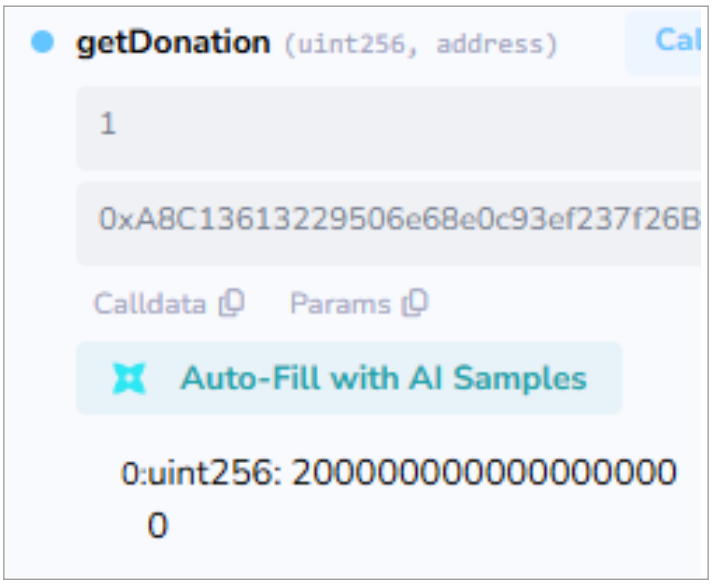


Figure 18: Donation verification by getDonation.

Observation: `getDonation` returns **2000000000000000000** wei (2 ETH) for campaign 1 and the queried donor address, so the contract recorded 2 ETH from this donor. (The brief's example of 0.1 ETH is not what these screenshots show.)

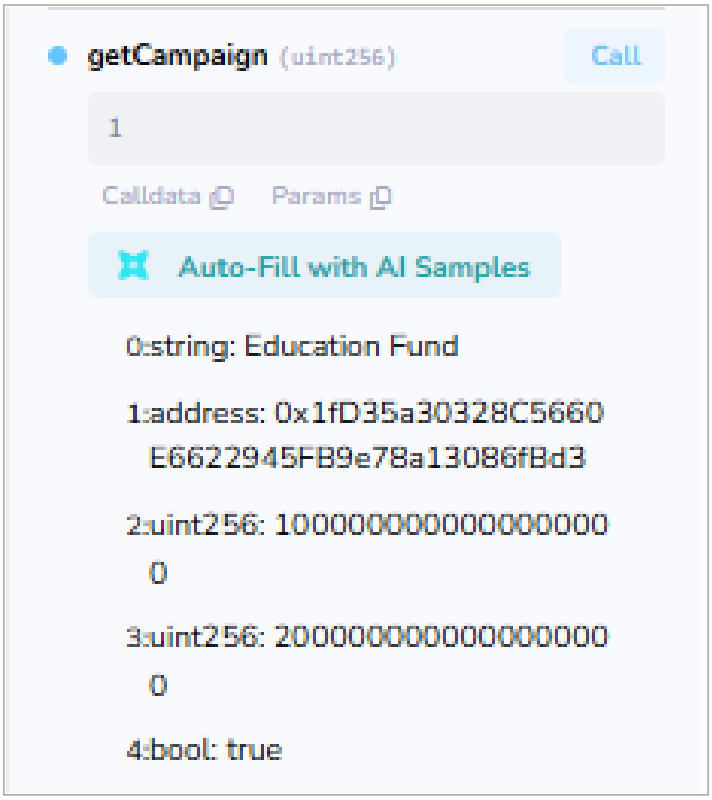


Figure 19: Campaign details returned by getCampaign after the donations.

Observation: Output: name **Education Fund**; beneficiary 0x1fD35a30328C5660E6622945FB9e78a13086fBd3; target **1000000000000000000** wei (1 ETH); amount raised **2000000000000000000** wei (2 ETH); active status **true**. The raised amount exceeds the target and the campaign still reports as active, so the displayed contract state does not close the campaign automatically at the target.

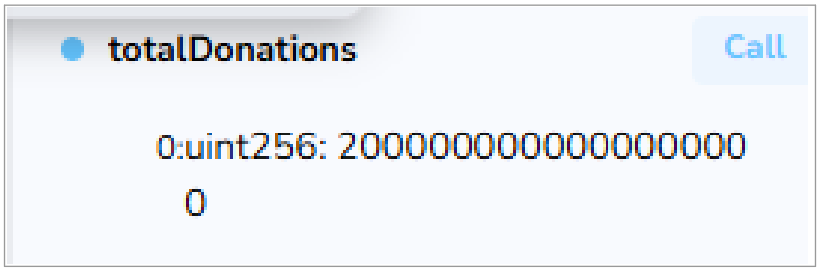


Figure 20: Public variable totalDonations.

Observation: `totalDonations` returns 2000000000000000000 wei (2 ETH), equal to the raised amount of campaign 1.

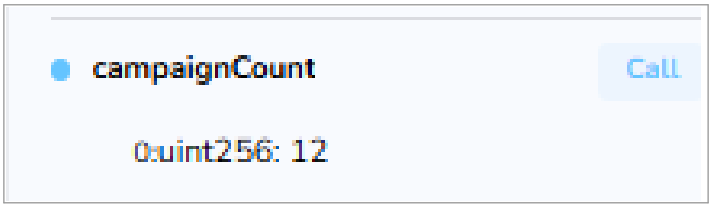


Figure 21: Public variable campaignCount.

Observation: `campaignCount` returns **12**. This suggests that campaign creation was run several times during testing; only campaign ID 1 (Education Fund) is displayed in the screenshots.

Step 11: Inspect contract-related transactions in Ganache

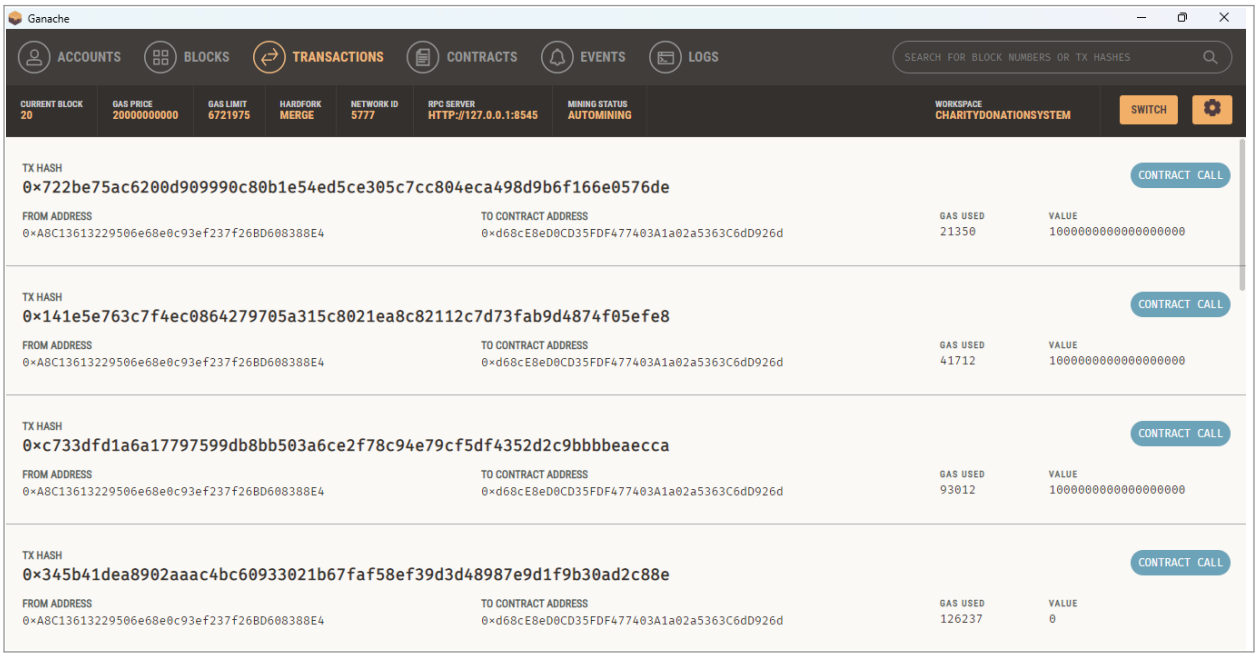


Figure 22: Transaction records displayed in Ganache.

Observation: At current block 20 (RPC server HTTP://127.0.0.1:8545, network ID 5777) the four most recent entries are **Contract Call** transactions from 0xA8C13613...88E4 to the contract 0xd68cE8eD...dD926d. Three carry a value of 1000000000000000000 wei (1 ETH) with gas used 21350, 41712 and 93012; the fourth has value 0 and gas used 126237, which fits a non-payable call such as campaign creation. The list does not show whether each call succeeded or reverted, and the contract records 2 ETH in total, so the status of each individual call should be verified in Ganache's transaction details.

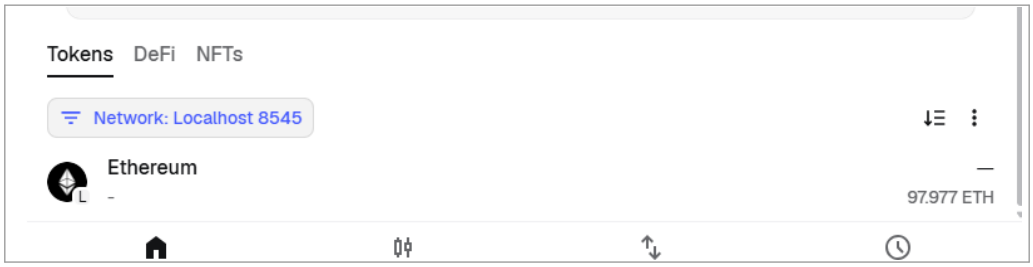


Figure 23: MetaMask balance after the contract interactions.

Observation: The Localhost 8545 balance of Imported Account 1 is now **97.977 ETH**, down from 100 ETH in Figure 10. This is consistent with Ether sent as donations plus gas fees, although the exact breakdown is not shown.

Step 12: Withdrawal function

No screenshot of `withdrawFunds` or of a withdrawal transaction was supplied, so withdrawal is explained in Step 12 only and is not claimed as demonstrated.

6. Result

Based on the supplied screenshots, a local Ethereum blockchain was started in Ganache (workspace CharityDonationSystem, network ID 5777, accounts funded with 100 test ETH), and the Ganache account 0xA8C1...88E4 was imported into MetaMask on the Localhost 8545 network. The contract **CharityDonation.sol** is shown compiled in Remix IDE and deployed to the local chain, with a contract-creation transaction recorded in Ganache (contract address 0xd68cE8eD...dD926d, gas used 1451523).

The **Education Fund** campaign (ID 1) was read back with `getCampaign`, showing a target of 1 ETH, 2 ETH raised and active status true. `getDonation` and `totalDonations` both return 2 ETH, and Ganache lists contract-call transactions to the contract address. The withdrawal function was not demonstrated, and the success of each individual transaction in the Ganache list is not established by the screenshots alone.

7. Conclusion

The experiment showed how a local blockchain (Ganache), a wallet (MetaMask), a browser IDE (Remix) and a Solidity smart contract can together form a transparent charity donation and fund-tracking system. Campaign data, donor contributions and totals are stored in the contract and can be read at any time, while each state-changing call appears as a transaction in Ganache with its sender, destination, gas used and value.

Through this practical I learned how to configure a local network in MetaMask, import a development account, connect Remix to a local node, compile and deploy a contract, create a campaign, and verify donations using `getCampaign`, `getDonation` and `totalDonations`. I also learned the difference between view calls and transactions, the role of gas, and how wei and Ether amounts are represented in contract inputs and outputs. The practical also showed that a contract must enforce its own rules (for example, the campaign remained active after exceeding its target), and that any such rule should be checked against the contract's full source code before relying on it.